

Experiment-9

Generative Adversarial Networks (GANs)

1. Aim

To implement a Deep Convolutional Generative Adversarial Network (DCGAN) on the MNIST dataset and observe adversarial training by comparing real and generated digit images over training epochs.

2. Theory

Generative Adversarial Networks (GANs), proposed by Goodfellow et al. in 2014, are a class of generative models used for learning the underlying data distribution and producing realistic synthetic samples. GANs serve as an alternative to Variational Autoencoders (VAEs) for modelling latent spaces, particularly in image generation tasks. Their key objective is to generate data that is statistically similar to real samples, making the generated outputs difficult to distinguish from actual data.

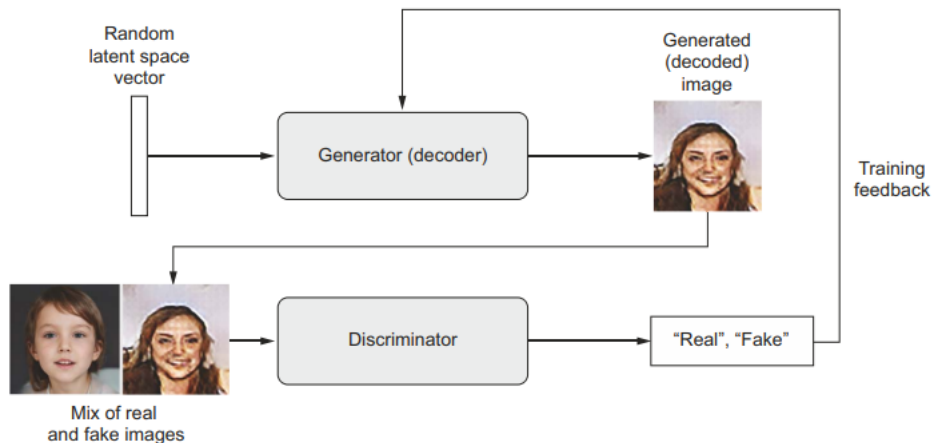


Fig. 1. A generator transforms random latent vectors into images, and a discriminator seeks to tell real images from generated ones. The generator is trained to fool the discriminator.

(Source: F. Chollet, *Deep Learning with Python*, 2nd ed., Manning Publications, 2021.)

A GAN consists of two neural networks trained simultaneously in a competitive framework.

- The generator network takes random noise vectors from a latent space as input and transforms them into synthetic images.
- The discriminator network receives both real images from the training dataset and fake images produced by the generator, and its task is to classify each input as real or generated.

The training process of a GAN can be intuitively understood as a competition between a counterfeit artist and an expert evaluator. Initially, the generator produces poor-quality images, which the discriminator easily identifies as fake. Through continuous feedback from the discriminator, the generator gradually improves its ability to create more realistic images. At the same time, the discriminator becomes more skilled at detecting subtle differences between real and generated samples. Over successive training iterations, both networks improve, eventually reaching a point where the discriminator finds it difficult to distinguish between real and synthetic images.

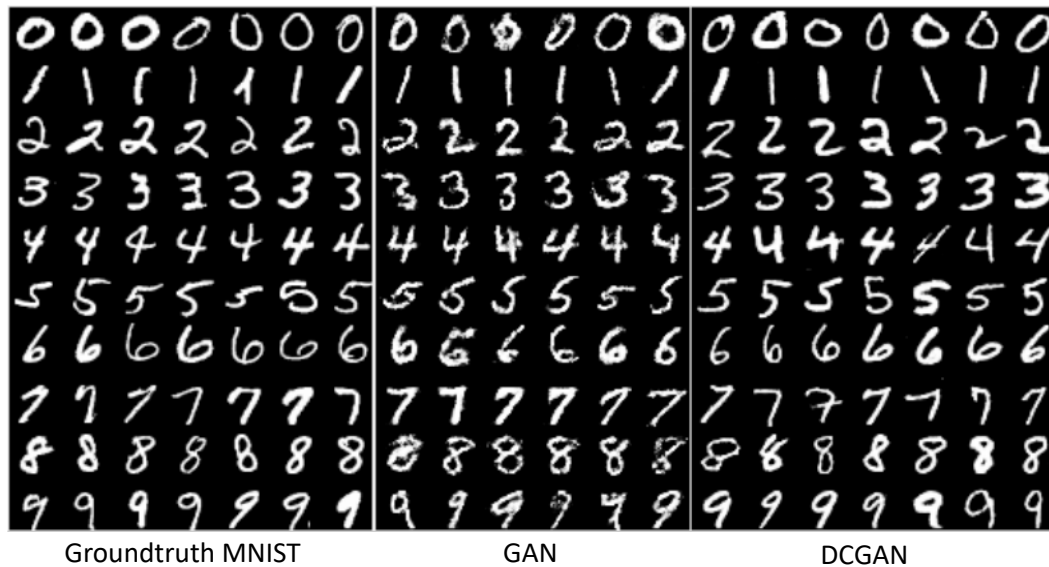


Fig. 2. Side-by-side illustration of (from left-to-right) the MNIST dataset, generations from a baseline GAN, and generations from our DCGAN

(Source: Radford, A., Metz, L., & Chintala, S., "Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks," arXiv:1511.06434, 2015.)

Adversarial Learning

The adversarial learning framework is easiest to understand when both the generator and discriminator are implemented using multilayer perceptrons. In this approach, the generator learns the data distribution by first taking random noise as input and transforming it into data-like samples using a neural network. This generator network maps random vectors to the data space and is trained to produce realistic outputs.

Along with the generator, a discriminator network is used, which is also a neural network that takes an input sample and outputs a single value representing the probability that the sample is real (from the training dataset) rather than generated. The discriminator is trained to correctly distinguish between real data and fake data produced by the generator.

During training, both networks are optimized together in an adversarial manner. The discriminator improves its ability to identify real and fake samples, while the generator improves its ability to fool the discriminator by producing more realistic outputs. This interaction can be viewed as a two-player game, where the generator tries to minimize the discriminator's ability to detect fake samples, and the discriminator tries to maximize its classification accuracy.

$$\min_{G,D} \max V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

where:

- x : A real data sample drawn from the training dataset
- $p_{data}(x)$: True data distribution
- z : Random noise vector sampled from a prior distribution

- $p_z(z)$: Prior noise distribution (usually Gaussian or uniform)
- $G(z; \theta_g)$: Generator network that maps noise z to a synthetic data sample
- θ_g : Parameters (weights) of the Generator
- $D(x; \theta_d)$: Discriminator network that outputs the probability that x is real
- θ_d : Parameters (weights) of the Discriminator
- $D(x) \in [0,1]$: Probability that input x is from real data

Meaning of Each Term:

- $E_{x \sim p_{data}(x)} [\log D(x)]$:
Encourages the discriminator to correctly classify real samples as real.
- $E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$:
Encourages the discriminator to correctly classify generated samples as fake, while the generator tries to minimize this term to fool the discriminator.

Interpretation:

- The Discriminator tries to maximize this value function by improving its ability to distinguish real from fake data.
- The Generator tries to minimize the function by generating samples that the discriminator classifies as real.

Unlike traditional deep learning models that optimize a fixed loss function, GAN training involves a dynamic optimization process. Each update to the generator affects the discriminator's learning objective and vice versa. Instead of converging to a single minimum, the training seeks an equilibrium between the two competing networks. This adversarial nature makes GANs powerful but also challenging to train, often requiring careful selection of network architectures, hyper-parameters, and training strategies.

Once training is complete, the generator can map any point from the latent space to a realistic output image. However, unlike VAEs, GANs do not explicitly enforce continuity or structure in the latent space, which can make interpolation and control more complex.

Merits of Generative Adversarial Networks:

- **High-Quality Data Generation:**
GANs are capable of generating high-resolution and highly realistic images, videos, and other types of data. The quality of the generated data is often superior to that produced by other generative models.
- **Unsupervised Learning:**
GANs can learn to generate data without requiring labelled training data. This is particularly useful in situations where labelled data is scarce or expensive to obtain.
- **Data Augmentation:**
GANs can generate synthetic data to augment existing datasets, which can be beneficial for training machine learning models, especially in scenarios where real data is limited.

Demerits of Generative Adversarial Networks:

- **Training Instability:**
Training GANs is notoriously difficult and unstable. The process often suffers from issues such as mode collapse, where the generator produces a limited variety of outputs, and vanishing gradients, where the discriminator becomes too strong, hindering the generator's learning.
- **Mode Collapse:**
Mode collapse is a common problem in GANs where the generator produces a narrow range of outputs, failing to capture the diversity of the data distribution. This can lead to poor generalization and limited applicability.
- **Susceptibility to Adversarial Attacks:**
GANs, like other neural networks, can be vulnerable to adversarial attacks, where small perturbations to the input data can significantly impact the output, potentially leading to misleading or harmful results.

3. Pre-Test (MCQs)

What is the primary application of Deep Convolutional GAN (DCGAN)?

a) Image-to-image translation

(Incorrect because image-to-image translation is typically handled by models like Pix2Pix or CycleGAN.)

b) Style transfer

(Incorrect because style transfer usually relies on CNN-based optimization or specialized GAN variants.)

c) High-resolution image generation

(Correct because DCGANs are designed to generate realistic images using convolutional layers, making them effective for image generation tasks.)

d) Anomaly detection

(Incorrect because anomaly detection uses GAN variants such as AnoGAN.)

Answer: c

2. Why is adversarial training used in GANs?

a) To train both networks independently

(Incorrect because GANs rely on dependent, not independent, training.)

b) To create competition between Generator and Discriminator

(Correct because this competition drives both networks to improve, resulting in more realistic generated samples.)

c) To speed up the training process

(Incorrect because adversarial training often makes training more unstable, not faster.)

d) To eliminate the need for labelled data

(Incorrect because while GANs are unsupervised, this is not the main reason for adversarial training.)

Answer: b

3. What is the purpose of noise input in the generator component of a GAN?

a) To randomize the generation process and introduce variations

(Correct because noise allows the generator to produce diverse outputs.)

b) To control the learning rate during training

(Incorrect because learning rate is a hyper-parameter, not controlled by input noise.)

c) To adjust the weights and biases of the generator

(Incorrect because weights are updated via backpropagation, not noise.)

d) None of the above

(Incorrect because noise plays a crucial role in sample diversity.)

Answer: a

4. What is the role of the discriminator in a GAN during the training process?

a) To provide feedback to the generator and help it improve

(Correct because it evaluates real vs. generated samples and its feedback guides the generator's learning.)

b) To generate synthetic samples

(Incorrect because sample generation is performed by the generator, not the discriminator.)

c) To adjust the learning rate during training

(Incorrect because learning rate is controlled by optimization algorithms, not the discriminator.)

d) None of the above

(Incorrect because the discriminator plays a critical evaluative role during GAN training.)

Answer: a

5. What does the term "adversarial" in GANs refer to?

a) The competitive process between two sub-models

(Correct because the generator and discriminator compete against each other.)

b) The generation of new data

(Incorrect because generation alone is not adversarial.)

c) The architecture of the generator

(Incorrect because adversarial refers to training dynamics, not architecture.)

d) The classification of real data

(Incorrect because classification is only one part of the process.)

Answer: a

6. What type of neural network layers are commonly used in DCGAN?

a) Recurrent layers

(Incorrect because DCGAN works on images, not sequences.)

b) Fully connected layers

(Incorrect because DCGAN minimizes dense layers.)

c) Convolutional layers

(Correct because DCGAN relies on convolution and transposed convolution layers.)

d) Max-pooling layers

(Incorrect because DCGAN uses strided convolutions instead.)

Answer: c

7. Which optimization algorithm is commonly used to train GANs?

a) Gradient Descent

(Incorrect because it converges slowly.)

b) Adam

(Correct because it provides adaptive learning rates and stable convergence.)

c) RMSprop

(Incorrect because it is less commonly used now.)

d) AdaGrad

(Incorrect because it decays learning rate too aggressively.)

Answer: b

8. Why are convolutional layers used in DCGAN instead of fully connected layers?

a) They require fewer parameters and capture spatial features

(Correct because convolutional layers efficiently learn spatial patterns in images while using fewer parameters than fully connected layers.)

b) They guarantee perfect image generation

(Incorrect because no architecture guarantees perfect generation.)

c) They eliminate the need for a discriminator

(Incorrect because the discriminator is essential in GAN training.)

d) They work only for grayscale images

(Incorrect because convolutional layers work for both grayscale and colour images.)

Answer: a

9. What does the “Real vs Generated” panel help in understanding?

a) Numerical accuracy of the model

(Incorrect because GANs do not use accuracy as a primary metric.)

b) Training speed of the optimizer

(Incorrect because training speed is unrelated to image comparison.)

c) Visual quality and realism of generated images

(Correct because it allows direct visual comparison to judge how realistic and similar the generated images are to real data.)

d) Memory usage of the GPU

(Incorrect because GPU memory usage is not reflected in image panels.)

Answer: c

10. What is the format and nature of images in the MNIST dataset used for GAN training?

a) RGB images of size 224×224

(Incorrect because MNIST images are not RGB and are much smaller than 224×224.)

b) Grayscale images of handwritten digits

(Correct because it consists of grayscale images representing handwritten digits (0–9).)

c) Colour images with multiple objects

(Incorrect because it contains single handwritten digits, not complex scenes.)

d) Text-based image representations

(Incorrect because it consists of image pixels, not textual encodings.)

Answer: b

4. Procedure

The objective of this procedure is to implement a Generative Adversarial Network (GAN) using a Deep Convolutional GAN (DCGAN) architecture on the MNIST dataset. The experiment involves training a generator and discriminator in an adversarial manner to learn the data distribution and generate digit-like images. The procedure focuses on understanding adversarial training dynamics, monitoring generator and discriminator losses, and visually comparing real and generated images across epochs using a “Real vs Generated” panel to evaluate model performance.

1. Environment Setup

Import the required Python libraries: *PyTorch*, *Torchvision*, *NumPy*, and *Matplotlib*.

2. Dataset Preparation

- Load the MNIST handwritten digit dataset.
- Apply pre-processing steps including resizing images to 16×16, converting them to tensors, and normalizing pixel values to the range $[-1,1]$.
- Separate the dataset into individual subsets for each digit (0–9).

- For each digit class, select a fixed number of samples and create separate DataLoaders to train the GAN independently on each digit.
3. Hyper-parameter Initialization
 - Define key hyper-parameters: latent vector size, number of training epochs per digit, learning rate, and Adam optimizer momentum term.
 - Set binary cross-entropy loss as the adversarial loss function.
 4. Model Architecture Definition
 - Design the Generator using transposed convolutional layers, batch normalization, and ReLU activations to transform random noise into digit images.
 - Design the Discriminator using convolutional layers with LeakyReLU activations to classify images as real or fake.
 - Apply weight initialization to both networks to ensure stable training.
 5. Training Strategy (Digit-wise GAN Training)
 - For each digit from 0 to 9:
 - Initialize a new Generator and Discriminator.
 - Assign Adam optimizers to both networks.
 - Cache a fixed batch of real images for visualization purposes.
 - Train the GAN for a fixed number of epochs using alternating optimization:
 - Update the Discriminator by minimizing loss on real and fake samples.
 - Update the Generator by minimizing its loss to fool the Discriminator.
 - Use label smoothing to improve training stability.
 6. Epoch-wise Monitoring and Visualization
 - Compute average Generator and Discriminator losses for each epoch.
 - Generate sample images using fixed noise vectors.
 - Display a “Real vs Generated” image panel at regular intervals to visually evaluate learning progress.
 - Assign a simple quality indicator based on smoothed generator loss values.
 7. Image Quality Evaluation
 - After each epoch, compute quantitative quality metrics:
 - Pixel diversity
 - Image sharpness
 - Image contrast
 - Store loss values and quality metrics for later analysis.
 8. Result Visualization
 - Load all trained generators and generate sample images for every digit.

- Display all generated digits together in a single grid for comparison.
9. Training Metrics Analysis
- Plot Generator and Discriminator loss curves for all digits.
 - Visualize image quality metrics across epochs to analyse training behaviour.
10. Final Training Summary
- Display final loss values and quality metrics for each digit.
 - Categorize training performance as Excellent, Good, or Needs Improvement.
11. Interactive Sample Generation
- Implement an interactive function to generate images for any selected digit.
 - Allow the user to specify the number of samples.
 - Display generated images along with corresponding quality metrics.
12. Result Analysis
- Generate samples for all digits to verify overall GAN performance.
 - Observe diversity and realism in the generated handwritten digits.

5. Post-Test (MCQs)

1. What are the two main components of a GAN?

a) Generator and encoder

(Incorrect because an encoder is not a core component of a standard GAN architecture.)

b) Discriminator and encoder

(Incorrect because GANs do not rely on encoders for adversarial training.)

c) Generator and discriminator

(Correct because the generator creates fake samples while the discriminator distinguishes real from fake.)

d) Encoder and discriminator

(Incorrect because the encoder is not involved in the basic GAN framework.)

Answer: c

2. How does the discriminator component in a GAN learn to distinguish between real and generated samples?

a) By minimizing the loss function of the generator

(Incorrect because the discriminator does not update the generator's loss.)

b) By maximizing the loss function of the generator

(Incorrect because the discriminator optimizes its own objective, not the generator's.)

c) By minimizing the loss function of the discriminator

(Correct because the discriminator learns by reducing its classification error between real and fake samples.)

d) By maximizing the loss function of the discriminator

(Incorrect because increasing the discriminator's loss would worsen its performance.)

Answer: c

3. Which loss function is commonly used in GANs?

a) Cross-entropy loss

(Incorrect because standard GANs specifically use a binary formulation rather than multi-class cross-entropy.)

b) Mean squared error loss

(Incorrect because MSE is mainly used in regression tasks, not binary discrimination.)

c) Binary cross-entropy (logistic loss)

(Correct because GANs frame real vs fake discrimination as a binary classification problem.)

d) Kullback-Leibler divergence

(Incorrect because KL divergence is not directly used in standard GAN loss functions.)

Answer: c

4. What is the purpose of the latent space in a GAN?

a) To represent the high-dimensional space of real samples

(Incorrect because the latent space is a lower-dimensional input space used to generate samples, not a direct representation of real data.)

b) To control the diversity and characteristics of generated samples

(Correct because different points in the latent space lead to different generated outputs, influencing variation and features.)

c) To adjust the learning rate during training

(Incorrect because learning rate control is handled by the optimizer, not the latent space.)

d) None of the above

Answer: b

5. How does the training process of a GAN typically work?

a) The generator and discriminator are trained alternately

(Incorrect because, although updates alternate in practice, the learning objective is defined as a simultaneous minimax game.)

b) The generator and discriminator are trained simultaneously

(Correct because GAN training is formulated as a two-player minimax optimization where both networks learn together.)

c) The generator is trained first, followed by the discriminator

(Incorrect because both models must co-evolve to improve generation quality.)

d) The discriminator is trained first, followed by the generator

(Incorrect because training only one network independently breaks the adversarial framework.)

Answer: b

6. Which approach is used to stabilize GAN training by controlling the learning rate of the Generator and Discriminator?

a) Learning rate decay

(Incorrect because it uniformly reduces the rate.)

b) Learning rate annealing

(Incorrect because it is less adaptive.)

c) Learning rate scheduling

(Correct because it dynamically adjusts learning rates during training.)

d) Learning rate adjustment

(Incorrect because it is too generic.)

Answer: c

7. In GANs, what happens when the discriminator becomes unable to distinguish real from fake samples?

a) Training stops

(Incorrect because training usually continues.)

- b) Generator updates its weights
(Correct because feedback still guides generator improvement.)
- c) Discriminator becomes the generator
(Incorrect because roles remain fixed.)
- d) Discriminator becomes more efficient
(Incorrect because inability implies reduced effectiveness.)

Answer: b

8. What is the purpose of the adversarial loss in GANs?

- a) To encourage the generator to produce samples that deceive the discriminator
(Correct because adversarial loss drives the generator to create realistic samples that the discriminator cannot distinguish from real data.)
- b) To control the learning rate during training
(Incorrect because the learning rate is set by the optimizer, not by the loss function.)
- c) To adjust the weights and biases of the generator
(Incorrect because weights and biases are updated via backpropagation, not directly by the loss definition.)
- d) None of the above

Answer: a

9. In which stage of GAN training does mode collapse occur?

- a) Early stage
(Incorrect because collapse usually develops later.)
- b) Middle stage
(Incorrect because it can still recover.)
- c) Late stage
(Incorrect because collapse is not restricted to a stage.)
- d) Throughout the training process
(Correct because mode collapse can occur at any point.)

Answer: d

10. Which technique can be used to address the problem of mode collapse in GANs?

- a) Gradient penalty
(Correct because gradient penalty stabilizes training and encourages smoother discriminator behaviour, helping reduce mode collapse.)
- b) Batch normalization
(Incorrect because batch normalization mainly stabilizes training but does not directly address mode collapse.)
- c) Dropout
(Incorrect because dropout is used for regularization and does not resolve mode collapse.)
- d) All of the above

Answer: a

6. References

1. Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y., "Generative Adversarial Nets," *Proceedings of the 27th International Conference on Neural Information Processing Systems (NeurIPS)*, Montreal, Canada, pp. 2672–2680, 2014
2. Radford, A., Metz, L., & Chintala, S., "Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks," arXiv preprint arXiv:1511.06434, 2015.

3. F. Chollet, *Deep Learning with Python*, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2021.